



Reverse 32-Bit Integer

Reverse the digits of a signed 32-bit integer. If the reversed integer overflows (i.e., is outside the range $[-2^{31}, 2^{31} - 1]$), return 0. Assume the environment only allows you to store integers within the signed 32-bit integer range.

Example 1:

Input: $n = 420$

Output: 24

Example 2:

Input: $n = -15$

Output: -51

Intuition

The primary challenge with this problem is in handling its edge cases. Before tackling these edge cases, let's first try handling the more basic cases and later see how we would need to modify our strategy.

Reversing positive numbers

Consider $n = 123$. Let's try building the reversed integer one digit at a time. The first thing to figure out is how to iterate through the digits of n to build our reversed number (initially set to 0):

$$n = 1\ 2\ 3, \quad \text{reversed_}n = 0$$

One way to do this is by starting at the last digit of n and appending each digit to **reversed_n**:

$n = 12$, $reversed_n = 32$


Let's explore how we can do this. To extract the last digit, we can use the modulus operator: $n \% 10$. This operation effectively finds what the remainder of n would be if divided by 10:

```
n = 123
digit = n % 10
      = 3
```

After extracting the last digit, we can remove it by dividing n by 10, which shifts the second-to-last digit to the last position, preparing it for the next iteration:

```
n = n // 10
  = 12
```

Once that's done, let's add the last digit extracted to our reversed number:

```
reversed_n += digit
           = 3
```

Below, we see the current states of n and $reversed_n$:

$n = 12$, $reversed_n = 3$

To process the next digit, let's extract it from n using the modulus operation, then remove it by dividing n by 10:

```
digit = n % 10
      = 2
n = n // 10
  = 1
```

To append this digit to $reversed_n$, we can multiply $reversed_n$ by 10 to shift its digits to the left, making space for the new digit. Then, we just add the new digit as before:

```
reversed_n = 10 * reversed_n + digit
           = 30 + 2
           = 32
```

We can repeat the above process until all digits of n are appended to $reversed_n$ (i.e., until n equals 0). Here's a breakdown of this process:

1. Extract the last digit: $digit = n \% 10$.

Reversing negative numbers

Before considering a separate strategy to handle negative numbers, let's first check if the set of steps above also work for negative numbers. Applying these steps to $n = -15$ gives:

$n = -15$, $reversed_n = 0$

```
digit = n % 10
      = -5

n = n // 10
  = -1

reversed_n = 10 * reversed_n + digit
            = -5
```

$n = -1$, $reversed_n = -5$

```
digit = n % 10
      = -1

n = n // 10
  = 0

reversed_n = 10 * reversed_n + digit
            = -51
```

As we can see, it works for negative numbers. Now, let's tackle situations in which reversing a number could result in integer overflow or underflow.

Detecting integer overflow

If the reverse of a positive number is larger than $2^{31} - 1$, it will overflow, and we should return 0. Let's call this maximum value **INT_MAX**.

$$\text{INT_MAX} = 2^{31} - 1 = 2147483647$$

Initially, it might seem sufficient to reverse the number completely, check if it exceeds $2^{31} - 1$, and return 0 if it does. However, in an environment where integers larger than $2^{31} - 1$ cannot be stored, attempting to reverse such an integer would cause an overflow:

$$1999999999 \xrightarrow{\text{reverse}} 9999999991 > \text{INT_MAX} \longrightarrow \text{overflow}$$

So, let's think of another way to detect overflow.

We're constructing the number **reversed_n** one digit at a time, which means we need to ensure not to cause the number to overflow with each new digit we add. Let's think about when adding a new digit might cause **reversed_n** to become too large. Here's how we can analyze this:

```

reversed_n = 21478364, digit = 7
append digit: reversed_n = reversed_n * 10 + digit
               = 214783647 == INT_MAX → no overflow

reversed_n = 21478364, digit = 8
append digit: reversed_n = reversed_n * 10 + digit
               = 214783648 > INT_MAX → overflow

```

Now, keep in mind that when `reversed_n == INT_MAX // 10`, only one more digit can be added to it. The key observation here is that this digit can only ever be 1 because a larger final digit would be impossible, as shown below:

```

      INT_MAX // 10
      ^
reversed_n = 2147483641 → implies n = 1463847412
      INT_MAX // 10
      ^
reversed_n = 2147483642 → implies n = 2463847412 > INT_MAX → impossible input

```

This means that when `reversed_n == INT_MAX // 10`, the last digit added to it won't cause an overflow, meaning we don't need to check the last digit in this case.

If `reversed_n` is already larger than `INT_MAX // 10`, adding any digit will cause it to overflow. We can handle this case with the following condition:

```
if reversed_n > INT_MAX // 10: return 0
```

Detecting integer underflow

We can apply similar logic to the above for handling integer underflow. Here, we just need to check that `reversed_n` never falls below `INT_MIN // 10`:

```
if reversed_n < INT_MIN // 10: return 0
```

Implementation

In Python, using the modulus operator (%) with a negative number gives a positive result. To avoid this, we can instead use **math.fmod** and cast its result to an integer to attain a negative modulus value.

For division, Python's // operator performs floor division, which can result in an undesired value when dealing with negative numbers (e.g., `-15 // 10` results in `-2`, instead of the desired `-1`). To achieve the desired behavior, use `/` for division and cast its result to an integer.

```
def reverse_32_bit_integer(n: int) → int:
    INT_MAX = 2**31 - 1
    INT_MIN = -2**31
    reversed_n = 0
    # Keep looping until we've added all digits of 'n' to 'reversed_n' in reverse
    # order.
    while n ≠ 0:
        # digit = n % 10
        digit = int(math.fmod(n, 10))
        # n = n // 10
        n = int(n / 10)
        # Check for integer overflow or underflow.
        if reversed_n > int(INT_MAX / 10) or reversed_n < int(INT_MIN / 10):
            return 0
        # Add the current digit to 'reversed_n'.
        reversed_n = reversed_n * 10 + digit
    return reversed_n
```

Complexity Analysis

Time complexity: The time complexity of `reverse_32_bit_integer` is $O(\log(n))$ because we loop through each digit of n , of which there are roughly $\log(n)$ digits. As this environment only supports 32-bit integers, the time complexity can also be considered $O(1)$.

Space complexity: The space complexity is $O(1)$.

☐ Mark as Complete

[Teach on ByteByteGo](#)

[Be an Affiliate](#)

[Become a Contributor](#)

hi@bytebytego.com

[Report a Bug](#)



[Newsletter](#)

[Privacy Policy](#)

[Terms of Service](#)

[Do Not Sell or Share My Personal Information](#)

[YouTube](#)